A Rust Implementation of the Cheng-Comley Certificateless Identity-Based Encryption Scheme

Khady Gaye

April 30, 2026

Abstract

Identity-Based Encryption (IBE) is an asymmetric encryption paradigm that allows one to encrypt a message with an arbitrary string as public key (e.g., an email address or phone number), with private keys generated a Key Generation Center (KGC). However, since the KGC possesses all private keys, IBE is inherently subject to the key-escrow problem. This led us to look for IBE systems free from this key escrow problem. Certificateless IBE is one such family, among others.

In this note, we implement the Cheng-Comley Certificateless Public-Key Encryption(CL-PKE) scheme in Rust. We choose the version of Cheng-Comley because they extend the scheme to an authenticated variant allowing the ciphertext to be tied not only to the identity of the recipient but also to the sender.

The scope of real-world deployments of CL-IBE systems remains very limited if not nonexistant. However, we wanted to implement the sheme using modern tools in the hope that it could eventually be used in real-world systems much like IBE is used in blockchain protocols. The choice of Rust and Arkworks is motivated by the fact that they provide implementations of elliptic curves with security guarantees suited to modern threat models. The scheme is IND-CCA2 secure; for full security proofs, we refer the reader to the original paper [1]. This note is intended as a practical reference for engineers and practitioners wishing to implement or study certificateless cryptography on modern pairing-friendly curves using `arkworks-rs`.

The full implementation is available at:
`https://github.com/Khady71/certificateless-ibe-on-arkworks`.

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Our Contribution . . . 3

2 Background and Notation **4**
2.1 Bilinear Pairings . . . 4
2.2 Type of pairings . . . 4
2.3 The Cheng–Comley CL-IBE Scheme . . . 4

3 Implementation Choices and Tricks **5**
3.1 Arkworks . . . 6
3.2 Curve Choice : BLS12-381 . . . 6
3.3 Type-1 to Type-3 Adaptation . . . 6
3.4 Objects . . . 6
3.5 Bit Security Estimate . . . 7
3.6 Hash Functions . . . 7

4 Implementation in Rust **8**
4.1 Tests and Test Vectors . . . 8
4.2 Performance . . . 9

5 Deployment Discussion **9**

1 Introduction

Traditional Public Key Infrastructure (PKI) requires a trusted Certificate Authority (CA) to bind public keys to identities, introducing significant infrastructure management overhead. IBE is very interesting because of its simplicity but it is inherent to the key-escrow problem. Several schemes have been proposed to address this barrier. Chow [12] in 2009, introduced a new entity called Identity-Certifying Authority (ICA) that authenticates users via certificates, allowing the KGC to anonymously generate secret keys. Emura et al. [13] in 2019, extended this approach adding a security layer and proposed a lattice-based variant targeting post-quantum security.

Certificateless public-key encryption(CL-PKE) offers an alternative solution to the key-escrow problem by splitting the private key in two-parts, one held by the KGC and one held by the user, both required for decryption. The paradigm was introduced by Al-Riyami and Paterson [2] in 2003. Cheng and Comley [1] proposed a more efficient construction in 2005 and extended it to an authenticated variant.

In the era of Zero-Knowledge Systems, CL-PKE can be useful besides addressing two well-known shortcomings of classical PKI (certificate management overhead) and traditional IBE (the key-escrow problem).

However, CL-PKE also removes one of the key advantages of IBE which is the ability to encrypt to a recipient who has not yet registered in the system. Indeed, in addition to splitting the private key in two parts, CL-PKE requires a second public key for encryption, distinct from the arbitrary identity string. The distribution of this additionnal introduced a new problem. But the challenge can be mitigated by system-level design choices. One approach would be to derive the second public key deterministically from the identity string, reducing the problem to a directory lookup. Another would be to notify the recipient to register before any encryption attempt, practical when the identity is an email address. Both approaches are application-specific and whether a general solution exists remains an open question to our knowledge.

1.1 Motivation

Privacy is the main driver behind the development of cryptographic methods. In modern applications, you need to trust an authority in order to achieve that privacy, which is a fundamental limitation we wanted to address. CL-IBE merges the simplicity of IBE with the freedom to rely on a trusted third party.

1.2 Our Contribution

To our knowledge, there is no publicly available open-source implementation of CL-IBE, unlike IBE which has several reference implementations. Our work bridges the gap between the theoretical scheme and a practical implementation. Concretely, we contribute the following:

- A full Rust implementation of the Cheng-Comley CL-PKE [1] scheme over BLS12-381 using `arkworks-rs 0.5.0`. A utility module implements helper functions for random sampling and hash function instantiation. The authenticated variant is not implemented and left as future work.
- An adaptation of the scheme from a Type-1 symmetric pairing to a Type-3 asymmetric pairing, with a detailed correspondence between the mathematical specification and the Rust implementation.
- Concrete parameter choices aligned with modern real-world cryptographic requirements.

- A discussion on the steps required to move from a paper-based scheme implementation to a production-ready deployment.

2 Background and Notation

2.1 Bilinear Pairings

To introduce bilinear pairings, we first consider the algebraic structures on which they are defined. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be cyclic groups of the same prime order p. The groups $\mathbb{G}_1$ and $\mathbb{G}_2$ are typically written additively, while $\mathbb{G}_T$ is written multiplicatively. These groups are usually instantiated as subgroups of elliptic curves and finite fields, chosen such that the discrete logarithm problem is hard in each of them.

Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be cyclic groups of the same prime order p. A bilinear pairing is a map

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

satisfying:

- **Bilinearity:** For all $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, and $a, b \in \mathbb{Z}_p$,

$$e(aP, bQ) = e(P, Q)^{ab}.$$

- **Non-degeneracy:** If P and Q are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, then $e(P, Q) \neq 1_{\mathbb{G}_T}$.
- **Efficiency:** The map e is efficiently computable.

2.2 Type of pairings

Pairings are commonly classified into three types:

- **Type-1 (symmetric):** $\mathbb{G}_1 = \mathbb{G}_2$.
- **Type-2:** $\mathbb{G}_1 \neq \mathbb{G}_2$ with an efficiently computable homomorphism $\phi : \mathbb{G}_2 \to \mathbb{G}_1$.
- **Type-3:** $\mathbb{G}_1 \neq \mathbb{G}_2$ with no efficiently computable homomorphisms between them.

The Cheng-Comley scheme and the original Boneh-Franklin IBE are both specified over Type-1 pairing. However the curve BLS12-381 from arkworks supports only Type-3 asymmetric pairing with no known isomomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$. Several works have studied the translation of Type-1 schemes into Type-3 as the latter is considered more efficient, more secure and require shorter keys to achieve modern security recommendation. Abe and al. [14] proposed a framework for the conversion where they build a dependency graph for each algorithm. A dependency graph describes how each group element appearing in the scheme depends on the other group element.

For our implementation, we followed a simpler approach used by reference IBE implementations notably `charm-crypto` [7] and `tlock` [8] which apply the same Type-1 to Type-3 conversion to Boneh-Franklin IBE. This approach does not alter the security proof of the scheme. The concrete conversion choices are detailed in Section 3.3.

2.3 The Cheng–Comley CL-IBE Scheme

We describe the certificateless IBE scheme of Cheng and Comley [1].

Algorithm 1 $\mathsf{Setup}(k)$

Generate cyclic groups $\mathbb{G}_1, \mathbb{G}_2$ of prime order q and a bilinear map $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$
Pick a random generator $P \in \mathbb{G}_1^*$
Sample $s \xleftarrow{\$} \mathbb{Z}_p^*$, compute $\mathsf{P_{pub}} \leftarrow sP$
Choose hash functions H_1, H_2, H_3, H_4
return $\mathsf{params} \leftarrow (q, \mathbb{G}_1, \mathbb{G}_2, \hat{e}, n, P, \mathsf{P_{pub}}, H_1, H_2, H_3, H_4)$, $\mathsf{msk} \leftarrow s$

Algorithm 2 $\mathsf{Extract}(\mathsf{params}, \mathsf{msk}, \mathsf{ID}_A)$

$Q_A \leftarrow H_1(\mathsf{ID}_A) \in \mathbb{G}_1^*$
$d_A \leftarrow s \cdot Q_A$
return d_A ▷ Sent to entity A over a secure channel

Algorithm 3 $\mathsf{Publish}(\mathsf{params}, t_A)$

Sample $t_A \xleftarrow{\$} \mathbb{Z}_p^*$
$N_A \leftarrow t_A \cdot P$
return N_A ▷ Published as entity A's public key

Algorithm 4 $\mathsf{Encrypt}(\mathsf{params}, \mathsf{ID}_A, N_A, m)$

Require: $m \in \{0,1\}^n$
Sample $\sigma \xleftarrow{\$} \{0,1\}^n$
$r \leftarrow H_3(\sigma, m)$
$Q_A \leftarrow H_1(\mathsf{ID}_A), \quad g^r \leftarrow \hat{e}(\mathsf{P_{pub}}, Q_A)^r, \quad f \leftarrow r \cdot N_A$
$U \leftarrow rP$
$V \leftarrow \sigma \oplus H_2(U, g^r, f)$
$W \leftarrow m \oplus H_4(\sigma)$
return $\mathsf{ct} \leftarrow \langle U, V, W \rangle$

Algorithm 5 $\mathsf{Decrypt}(\mathsf{params}, d_A, t_A, \mathsf{ct})$

Require: $\mathsf{ct} = \langle U, V, W \rangle$
$g' \leftarrow \hat{e}(U, d_A)$
$f' \leftarrow t_A \cdot U, \quad \sigma' \leftarrow V \oplus H_2(U, g', f')$
$m' \leftarrow W \oplus H_4(\sigma')$
$r' \leftarrow H_3(\sigma', m')$
if $U \neq r'P$ **then return** $\perp$
end if
return m'

The key-escrow problem is avoided because decryption requires both d_A (from the KGC) and t_A (known only to the user).

3 Implementation Choices and Tricks

The implementation is written taking inspiration from existing open-source IBE Github repository on the same library and on charm-crypto. Specifically, we drew from the Waters-Naccache IBE

implementation by the encryption4all project [6], itself based on the scheme by Naccache [10] which reduces the public key size of Waters' standard-model IBE [11]. We also referenced the Charm implementation of the same scheme [7]. For pairing and curve arithmetic, we took inspiration from code examples in the arkworks documentation and the github repository [9], which demonstrates BLS (Boneh-Lynn-Shacham) signatures [BLS01] on arkworks, a pairing-based signature scheme widely deployed in blockchain consensus protocols. Finally, we draw from [8], which implements verifiable timelock encryption on BLS12-381 using arkworks circuits, combining IBE with zero-knowledge proofs to encrypt against a future threshold network round. We use an analogy between each reference scheme and our own construction, adapting the Type-3 pairing layout consistently across all implementations.

3.1 Arkworks

Arkworks is mostly used in zero-knowledge SNARKS systems. It provides elliptic curves implementations and interfaces for pairing operations. Elliptic curve points can be represented in affine coordinates(x,y) or projective ones(X : Y : Z). The projective representation being more efficient in arithmetic, we use it for all computations and affine coordinates for output and storing elements since they provide a unique representative for a curve point.

3.2 Curve Choice : BLS12-381

The main aim was to use an elliptic curve used by production-ready systems. BLS (Barreto-Lynn-Scott) are a family of pairing-friendly elliptic curves. BLS12-381 is a construction from the family, 12 is the embedding degree of the curve and 381 is the size of the key and the number of bits needed to represent coordinates on the curve.

3.3 Type-1 to Type-3 Adaptation

In this work, we use a Type-3 pairing instantiated over the BLS12-381 curve, where $\mathbb{G}_1 \subset E(\mathbb{F}_q)$, $\mathbb{G}_2 \subset E'(\mathbb{F}_{q^2})$, and $\mathbb{G}_T$ is a multiplicative subgroup of $\mathbb{F}^*_{q^{12}}$. Since pairing in the group can only be asymmetric, one the elements should be in G1 and the second in G2, the result of pairing being stored in GT. There are two pairing operations in the scheme : the first one $\hat{e}(P_{pub}, Q_A)$, where $P_{pub} = sP$ is derived from the system generator and $Q_A = H_1(ID_A)$ is obtained by hashing the recipient's identity to a curve point. This choice follows the same assignment used in reference IBE implementations on BLS12-381 [8, 7], from which our scheme is derived and which share the same pairing structure.

The remaining group elements follow naturally from this assignment. Elements derived from P via scalar multiplication, such as $N_A = t_A P$ and $U = rP$, stay in $\mathbb{G}_1$. The partial private key $d_A = sQ_A$ stays in $\mathbb{G}_2$. Scalar values including the master secret s, user secret t_A, and derived scalar r live in the scalar field $\mathbb{F}_r$. The pairing output $g^r = \hat{e}(P_{pub}, Q_A)^r$ lands in $\mathbb{G}_T$. Scalar multiplication is closed within each source group and requires no adaptation.

3.4 Objects

For clarity purposes, taking inspiration from the repository encryption4all [6] and for future integrations, all scheme objects are encapsulated into typed Rust structs. The public key groups the system generator P and master public key P_{pub} into a `PublicKey` struct. The ciphertext components $\langle U, V, W \rangle$ are grouped into a `CipherText` struct. Keys are split according to their ownership: `UserSecretKey` holds the partial private key $d_A \in \mathbb{G}_2$ known to both the KGC and the user, while `UserPrivateKey` holds the user-chosen secret $t_A \in \mathbb{F}_r$ known only by the user. This separation reflects the certificateless structure of the scheme directly in the type system, making misuse harder by construction.

The implementation is organised into two files. The `utils` module implements random sampling and the four hash functions. The main module implements the five core algorithms of the scheme, calling utilities from `utils`. All types use arkworks' native representations: $\mathbb{G}_1$ and $\mathbb{G}_2$ elements are stored in affine coordinates (`G1Affine`, `G2Affine`) and scalar field as elements of `Fr`.

3.5 Bit Security Estimate

According to the `ark-bls12-381` documentation [5], BLS12-381 is defined over a 381-bit prime field. It was designed to replace BN254 in order to provide a higher security level without incurring a large performance overhead. The curve provides approximately 128 bits of security against discrete logarithm attacks in $\mathbb{G}_1$ and $\mathbb{G}_2$, and against the decisional Bilinear Diffie-Hellman (BDHP) problem on which the security of our scheme relies.

3.6 Hash Functions

There are four hash functions needed for the whole scheme. Table 1 summarizes the instantiation of the four random oracles required by the scheme. All functions use distinct domain separation tags (DST) to prevent cross-oracle substitution attacks.

Table 1: Hash function instantiation on BLS12-381

Oracle	**Scheme signature**	**Construction**	**Output**
H_1	$\{0,1\}^* \to \mathbb{G}_2^*$	RFC 9380	$\mathbb{G}_2$ point
H_2	$\mathbb{G}_1 \times \mathbb{G}_T \times \mathbb{G}_1 \to \{0,1\}^n$	SHA-256 (compressed serialization)	32 bytes
H_3	$\{0,1\}^n \times \{0,1\}^n \to \mathbb{Z}_q^*$	SHA-256 + `from_le_bytes_mod_order`	`Fr` scalar
H_4	$\{0,1\}^n \to \{0,1\}^n$	SHA-256	32 bytes

H_1 — **Hash-to-curve.** H_1 maps an arbitrary identity string to a point in $\mathbb{G}_2^*$. Since the original scheme uses a Type-1 pairing with $H_1 : \{0,1\}^* \to \mathbb{G}_1^*$. The construction follows RFC 9380 [15], an IETF standard that specifies how to deterministically map arbitrary byte strings to elliptic curve points in a way that is indifferentiable from a random oracle. Concretely, we use the `MapToCurveBasedHasher` trait, whose `hash()` method encapsulates the full RFC 9380 pipeline.

H_2 — **Hash to bitstring.** H_2 takes a tuple $(U, g^r, f) \in \mathbb{G}_1 \times \mathbb{G}_T \times \mathbb{G}_1$ and produces an n-byte mask. Since these inputs are algebraic objects of different types, they must first be serialized to a canonical byte representation before hashing. We use compressed serialization [5]: each $\mathbb{G}_1$ point is encoded as a single field element (48 bytes on BLS12-381) with a sign bit, and the $\mathbb{G}_T$ element as a compressed extension field element. The concatenated bytes are then processed by SHA-256 under a dedicated DST.

H_3 — **Hash to scalar.** H_3 maps $(\sigma, m) \in \{0,1\}^n \times \{0,1\}^n$ to a scalar $r \in \mathbb{Z}_q^*$. The two inputs are fed sequentially to SHA-256 via `hasher.update()`, and the 256-bit digest is interpreted as a scalar via `from_le_bytes_mod_order`, which reduces the digest modulo the BLS12-381 group order q. This introduces a statistically negligible bias, the same approach used in `tlock-rs` [8], which is acceptable in practice given the proximity of 2^{256} and q.

H_4 — **Hash to bitstring.** H_4 masks the plaintext by hashing σ under a dedicated DST, following the same construction as H_2 with a single input. Domain separation between H_2 and H_4 is enforced at the DST level, ensuring that identical inputs to both oracles produce independent outputs.

4 Implementation in Rust

We describe the correspondence between scheme operations and code implementation in Table 2. We will be describing in the table only the setup and encrypt algorithm operations since they capture almost all of the operations.

Scheme	Description	Rust / arkworks	Note
Groups and fields			
$\mathbb{G}_1$	Source group	`G1Affine` / `G1Projective`	BLS12-381
$\mathbb{G}_2$	Source group	`G2Affine` / `G2Projective`	Type-3 adaptation
$\mathbb{G}_T$	Target group	`PairingOutput<Bls12_381>`	
$\mathbb{Z}_q^*$	Scalar field	`Fr`	
Keys			
$P \in \mathbb{G}_1$	System generator	`PublicKey.p: G1Affine`	Random in $\mathbb{G}_1$
$s \in \mathbb{Z}_q^*$	Master secret	`MasterSecretKey.s: Fr`	
$P_{pub} = sP$	Master public key	`PublicKey.p_pub: G1Affine`	`p * s`
$t_A \in \mathbb{Z}_q^*$	User secret value	`UserSecretKey.t_a: Fr`	
$N_A = t_A P$	User public key	`UserPublicKey.n_a: G1Affine`	
$d_A = sQ_A$	Partial private key	`PartialKey.d_a: G2Affine`	
Encrypt operations			
$\sigma \xleftarrow{\$} \{0,1\}^n$	Random seed	`rand_sigma(rng): [u8;32]`	
$r = H_3(\sigma, m)$	Derived scalar	`let r: Fr = h3(&sigma, &message)`	
$Q_A = H_1(ID_A)$	Identity point	`h1_to_g2(id): G2Affine`	
$g^r = \hat{e}(P_{pub}, Q_A)^r$	Pairing element	`Bls12_381::pairing(p_pub, q_a) * r`	
$f = rN_A$	Blinding term	`upk.n_a * r : G1`	
$U = rP$	Ciphertext elem.	`pk.p * r : G1`	
$V = \sigma \oplus H_2(U, g^r, f)$	Masked seed	`sigma[i] ^ h2_result[i]`	
$W = m \oplus H_4(\sigma)$	Masked message	`message[i] ^ h4_result[i]`	
$C = \langle U, V, W \rangle$	Ciphertext	`CipherText{u, v, w}`	

Table 2: Correspondence between Cheng-Comley CL-PKE notation and Rust implementation

4.1 Tests and Test Vectors

We implement the following unit tests using `cargo test`:

- **Round-trip**: `encrypt` followed by `decrypt` returns the original message, verifying the basic correctness of the scheme.
- **Wrong identity**: decrypting with a different identity returns $\perp$, verifying that the ciphertext is bound to the intended recipient. $\perp$ denotes a failure or rejection output
- **Wrong user secret**: decrypting with a different x returns $\perp$, verifying that the user secret is required for successful decryption.
- **Hash determinism**: all four hash functions return the same output when called twice on the same input.
- **Hash domain separation**: H_i and H_j with $i \neq j$ produce different outputs on the same input, ensuring that distinct oracles cannot be substituted for one another.

4.2 Performance

Benchmarks measuring the computational cost of each algorithm are left as future work. A timing evaluation using the `criterion` crate is planned for a subsequent version of this implementation.

5 Deployment Discussion

The current implementation covers the core cryptographic algorithms and passes all correctness tests, but several steps remain before it can be considered production-ready.

The most immediate next step is to implement the authenticated variant of the scheme, in which the ciphertext is bound to the sender's identity, as originally proposed by Cheng and Comley [1].

On the performance side, benchmarks using the `criterion` crate are planned to measure the cost of each algorithm, in particular the pairing operations which dominate the runtime.

A deployment-ready version would require a comprehensive security audit covering memory safety, constant-time execution, and correct serialization. CL-PKE is not currently standardized by bodies such as NIST or IETF, which constitutes an additional barrier to adoption in regulated environments. In practice, its use in a private SaaS setting is more realistic when combined with well-established primitives (e.g., symmetric encryption such as AES for bulk data protection), and supported by thorough security evaluation.

Another practical challenge lies in the public key distribution inherent to CL-PKE, namely how the recipient's public value N_A is made available to the sender prior to encryption. One possible approach is to derive N_A deterministically from the identity string, reducing the problem to a directory lookup.

Overall, this work demonstrates the feasibility of certificateless encryption on modern pairing-friendly curves and represents a step toward practical systems that reduce reliance on a single trusted key authority.

Acknowledgments

References

[1] Z. Cheng and R. Comley. Certificateless public key encryption without pairing. *IACR ePrint*, 2005/012, 2005. https://eprint.iacr.org/2005/012

[2] S. Al-Riyami and K. Paterson. Certificateless public key cryptography. In *ASIACRYPT 2003*, LNCS 2894, pp. 452–473.

[3] D. Boneh and M. Franklin. Identity-based encryption from the Weil pairing. In *CRYPTO 2001*, LNCS 2139, pp. 213–229.

[4]

[5] The arkworks contributors. arkworks-rs: A Rust ecosystem for zkSNARKs. https://arkworks.rs

[6] encryption4all/ibe: Reference IBE implementations in Rust. https://github.com/encryption4all/ibe

[7] charm/blob/dev/charm: Reference IBE implementations in Charm-Crypto. https://github.com/JHUISI/charm/blob/dev/charm/schemes/ibenc

[8] nulltea/zk-timelock: Arithmetic circuits for verifiable time-lock encryption. https://github.com/nulltea/zk-timelock

[9] ArnaudBrousseau/bls-on-arkworks: This crate implements the latest IRTF draft for BLS signatures. https://github.com/ArnaudBrousseau/bls_on_arkworks

[10] D. Naccache Secure and Practical Identity-Based Encryption. https://eprint.iacr.org/2005/369.pdf

[11] B. Waters Efficient Identity-Based Encryption Without Random Oracles. https://eprint.iacr.org/2004/180.pdf

[12] Sherman S.M. Chow Removing Escrow from Identity-Based Encryption https://www.iacr.org/archive/pkc2009/54430261/54430261.pdf

[13] Keita Emura Identity-Based Encryption with Security against the KGC: A Formal Model and Its Instantiations https://eprint.iacr.org/2019/1384.pdf

[14] M. Abe, J. Groth, M. Ohkubo, T. Tango Converting Cryptographic Schemes from Symmetric to Asymmetric Bilinear Groups https://doi.org/10.1007/978-3-662-44371-2_14

[15] A. Faz-Hernandez, S. Scott, N. Sullivan, R. Wahby, C. Wood. Hashing to Elliptic Curves. RFC 9380, Internet Engineering Task Force, 2023. https://www.rfc-editor.org/rfc/rfc9380